

VFEAgent: A Multimodal Agent Framework for End-to-End Automated Finite Element Analysis

Jiachen Zhang^{1,2,*}, Junyi Lao^{1,*}, Chenghao Liu^{1,*}, Siyuan Liu¹,
Shixin Wu¹, Linsen Zhang¹, Boyu Wang¹ and Songfang Huang^{1,†}

¹Peking University

²China Agricultural University

Abstract

Finite Element Analysis (FEA) serves as the cornerstone of modern engineering design. However, its workflow is inherently complex and relies heavily on domain expertise. Although recent efforts have integrated Large Language Models (LLMs) into FEA, existing approaches face limitations in handling multimodal inputs and executing complex tasks. To address these limitations, we propose VFEAgent, an end-to-end multi-agent system designed to automate FEA modeling and simulation directly from input images and problem descriptions.

Our methodology integrates two core components: (1) a multimodal vision-language multi-agent pipeline that employs ReAct-driven reasoning to extract structured FEA specifications from heterogeneous inputs and (2) a verification-first code synthesis framework, incorporating robust self-debugging and fallback mechanisms to ensure executability and physical validity. We systematically evaluated the system across various engineering mechanics scenarios. The results demonstrate that VFEAgent achieves a high success rate in generating complete and physically valid simulations, outperforming LLM-based baseline methods in reliability and correctness. These findings validate the feasibility of automating the complete FEA workflow, highlighting the framework’s potential to liberate engineers from tedious manual analysis.

1 Introduction

Structural engineering analysis is the cornerstone of ensuring the safety and reliability of large-scale infrastructure, aerospace vehicles, and complex mechanical assemblies, directly determining the scientific validity of engineering decisions. As the primary methodology of computational mechanics, Finite Element Analysis (FEA) has fundamentally transformed the paradigm of structural analysis by discretizing continuous media into numerical elements [Liu *et al.*,

2022]. However, despite the formidable power of contemporary computational engines, the current FEA workflow remains characterized by high empirical dependency and intensive human-computer interaction [Sahani, 2025]. This necessitates that expert engineers manually bridge the chasm between engineering drawings, geometric modeling, and physical environment configuration [Azanaw, 2025]. Such a process is not only labor-intensive and time-consuming but also highly susceptible to human error during tedious parameter configuration and topological processing [Shah *et al.*, 2024]. In the context of modern engineering demanding high-frequency iterations, this over-reliance on manual operation constrains the deep integration of digital design and intelligent manufacturing.

The rapid ascent of Large Language Models (LLMs) and Vision-Language Models (VLMs) offers a transformative opportunity to automate the FEA pipeline [Baker *et al.*, 2025]. By leveraging the scripting interfaces of mainstream commercial software (e.g., the Abaqus Scripting Interface/API), Multi-Agent Systems (MAS) can theoretically translate high-level engineering intents into precise instruction sequences [Ni and Buehler, 2024; Hong *et al.*, 2024]. Nevertheless, existing AI-integrated FEA frameworks face three critical limitations that hinder their practical utility [Sahani, 2025]. Most fundamentally, **existing works predominantly rely on hard-coded pipelines or preset knowledge bases**. Instead of performing genuine *de novo* geometric construction, these systems execute “pseudo-modeling” by populating parameters into prototypes retrieved from pre-defined libraries [Qi *et al.*, 2025]. Such template-matching fails to satisfy the vast majority of real-world design requirements, which demand precise, customized parametric representations of non-standard structures under non-fixed operating conditions [Göpfert *et al.*, 2023]. Compounding this issue, **current agents are often vision-blind**. By utilizing pre-processed structured text or meshes as input, they circumvent the most challenging phase of FEA—the interpretation of raw blueprints—thereby losing high-fidelity semantic information [Khan *et al.*, 2024]. Furthermore, **feedback mechanisms remain immature**. Existing debugging strategies are largely confined to shallow syntax errors, lacking the ability to identify implicit physical logic discrepancies or form a closed-loop self-healing capability that integrates long-term experience with short-term reflection [Ghorbani *et al.*, 2024;

*These authors contributed equally to this work.

†Corresponding author *E-mail address*: hsf@pku.edu.cn

Shinn *et al.*, 2024].

To address these gaps and explore the profound potential of AI in autonomous engineering, we propose VFEAgent, an end-to-end framework validated on the commercial software Abaqus [Hou *et al.*, 2025]. VFEAgent is the first autonomous FEA framework capable of transforming raw engineering drawings into executable simulation scripts and computational results [Qi *et al.*, 2025]. The system is driven by two synergistic components: FEM_VLM_MAS, which employs a ReAct-driven hierarchical perception strategy to parse blueprints, and Text2Abaqus, an LLM-driven engine that generates and executes simulation scripts [Ni and Buehler, 2024; Yao *et al.*, 2023; Wei *et al.*, 2022]. Unlike prior pseudo-modeling attempts, VFEAgent decodes visual signals into structured engineering semantics and synthesizes models from the ground up [Khan *et al.*, 2025]. To bridge the gap in specialized evaluation, we further introduce a graded expert-curated, vision-augmented benchmark, containing 15 complex cases—ranging from asymmetrical steel frames and beam topology optimization to pressure vessels and metamaterials—to evaluate core physical quantities such as Mises stress, modal frequencies, acceleration, and strain energy [Mohammadzadeh *et al.*, 2025]. The main contributions of this paper are as follows:

- **Realization of the first genuinely end-to-end FEA multi-agent system.** Diverging from retrieval-based pseudo-modeling, VFEAgent achieves full-process autonomy—from raw image input to geometric construction, physical solving, and post-processing—demonstrating strong universality for non-standard, complex industrial scenarios.
- **A Hierarchical Perception Mechanism.** By simulating expert habits, FEM_VLM_MAS system utilizes a multi-level extraction strategy to decode engineering drawings. It progressively decodes engineering drawings from numerical values and nodes to load distributions and connectivity, enabling precise transformation from visual signals to structured engineering semantics.
- **Self-Healing Script Generation Engine.** By introducing a debugger that fuses short-term reflection with long-term memory, Text2Abaqus system enables closed-loop rectification of both syntax errors and physical logic biases, significantly enhancing the reliability of autonomous simulation.
- **Release of a Graded Solid Mechanics Evaluation Framework.** We provide an expert-validated vision-augmented benchmark featuring complex physical scenarios and multi-dimensional metrics. This framework serves as a quantitative baseline for evaluating the engineering design thinking and constraint-processing capabilities of large models.

2 Related Work

2.1 LLM-Integrated FEA

Early research on integrating Large Language Models (LLMs) into Finite Element Analysis (FEA) primarily focused on automating individual stages of the simulation

lifecycle to reduce manual labor. In geometry and meshing, NekMesh [Green *et al.*, 2024] introduced a CAD-independent framework for high-order mesh generation using variational untangling and octree-based adaptive sampling. For expert-level decision support, Gembarski [Gembarski, 2020] proposed a multi-agent system (MAS) that simulates expert negotiations to identify model features and manufacturing constraints. Recently, frameworks like FRAME [Guru *et al.*, 2025] and the work by Tian *et al.* [Tian and Zhang, 2024] explored “self-optimization” workflows that identify physical issues, such as stress concentration, and automatically trigger geometric modifications. However, these partial assistants typically require structured inputs and human intervention to bridge the chasm between disparate simulation phases.

The field has recently evolved toward end-to-end autonomous agents that orchestrate the full “Geometry-Mesh-Simulation-Analysis” (GMSA) pipeline. MechAgents [Ni and Buehler, 2024] utilized role-playing agents to solve elasticity problems using FEniCS, while FeaGPT [Qi *et al.*, 2025] implemented the first conversational GMSA pipeline driven by natural language. Furthermore, Geng *et al.* [Geng *et al.*, 2025] proposed a lightweight MAS framework to automate 2D frame modeling by decoupling structural analysis into specialized tasks. In fluid mechanics, ChatCFD [Fan *et al.*, 2025] leveraged structural reasoning to correct complex coupled errors in compressible flows. Despite these advancements, existing frameworks heuristically bypass core challenges through “pseudo-modeling” strategies. Systems such as MooseAgent [Zhang *et al.*, 2025] and AutoFEA [Hou *et al.*, 2025] rely heavily on Retrieval-Augmented Generation (RAG) to fill parameters into predefined templates rather than performing genuine *de novo* construction. Furthermore, these models remain largely “vision-blind,” bypassing the critical interpretation of raw engineering blueprints and struggling with deep physical logic, such as non-linear plasticity.

2.2 Evaluation Benchmarks

Evaluation benchmarks have similarly lagged behind the demands of autonomous engineering. Current benchmarks primarily focus on either commercial API operation, such as FEABench [Mudur *et al.*, 2025], or underlying mathematical implementation, such as FEM-BENCH [Mohammadzadeh *et al.*, 2025]. While EngDesign [Guo *et al.*, 2025] introduced design-thinking metrics, its structural tasks are confined to idealized 2D geometries like simple trusses or rectangular beams. Overall, existing benchmarks lack coverage of non-standard industrial topologies and fail to assess the visual reasoning required to parse raw drawings. Moreover, they often focus on code syntax rather than high-fidelity physical field distributions, such as modal frequencies or strain energy. Consequently, a gap remains between passing synthetic code tests and solving real-world, vision-augmented engineering problems.

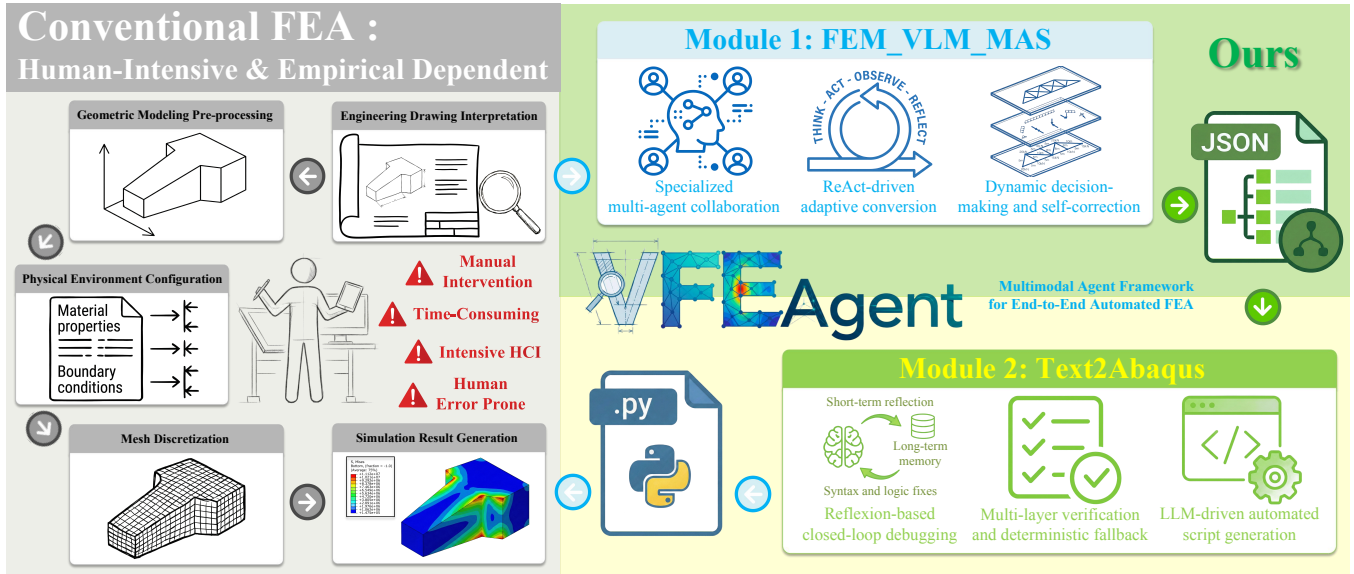


Figure 1: VFEAgent automates the transformation of engineering drawings into validated finite element simulation results.

3 Methodology

3.1 Problem Formulation

We define the automated FEA modeling task as a mapping from a multimodal input tuple $\mathcal{X} = (I, T_{ctx})$ to a valid physical response field $\mathcal{R}$. Here, I denotes the structural diagram and T_{ctx} contains textual constraints (e.g., material properties, load magnitudes). To ensure physical validity and software agnosticism, we factorize this end-to-end process into an intermediate grounded state Y :

$$\mathcal{X} \xrightarrow{\phi_{\text{perc}}} Y \xrightarrow{\phi_{\text{syn}}} S \xrightarrow{\text{Solver}} \mathcal{R} \quad (1)$$

where $Y = (\mathcal{G}, \mathcal{M}, \mathcal{BC}, \mathcal{L})$ represents a *solver-agnostic intermediate representation* (IR) enforced by a strict schema. Specifically, $\mathcal{G} = \{V, E\}$ denotes the geometry topology (nodes and connectivity), $\mathcal{M}$ specifies material sections, while $\mathcal{BC}$ and $\mathcal{L}$ define boundary conditions and load vectors mapped to $\mathcal{G}$. S is the executable simulation script (e.g., Abaqus Python) generated from Y .

3.2 System Architecture

The **VFEAgent** framework implements the mappings ϕ_{perc} and ϕ_{syn} via two coupled modules. **Stage-A (FEM_VLM_MAS)** employs a multi-agent ReAct system to instantiate Y from visual inputs. **Stage-B (Text2Abaqus)** implements a verification-driven synthesis loop with a neuro-symbolic handover protocol to guarantee the executability of S .

3.3 Stage-A: ReAct-Driven Semantic Perception

Multi-Agent Orchestration. We decompose the complex visual reasoning task into four specialized roles: (1) *Perception Agent* for OCR and geometric primitive detection; (2) *Reasoning Agent* for inferring topological connectivity and component types; (3) *Validation Agent* for auditing schema consistency; and (4) *Orchestrator* for global planning. The

Orchestrator maintains a dynamic belief state B_t and executes a decision loop: $a_t = \pi(B_t, \text{Prompt}_{ctx})$, where actions a_t include invoking sub-agents or updating the schema.

Visual Perception Mechanism

End-to-End Direct Reasoning. Unlike pipeline approaches that rely on intermediate pre-processing (e.g., Canny edge detection or grid overlays), our Perception Agent directly feeds the raw engineering diagram into the VLM. This end-to-end strategy leverages the model’s internal world knowledge to resolve visual ambiguities—such as distinguishing a “dimension line” from a “structural beam”—that heuristic CV algorithms often misclassify.

Constraint-Aware Coordinate Estimation. Extracting precise coordinates from unscaled raster images is ill-posed from a physical perspective. VFEAgent addresses this via a *semantic estimation* strategy. When dimension annotations are present, the VLM infers the pixel-to-metric scale to establish the global coordinate system. In the absence of explicit dimensions, the system prioritizes *topological consistency* (relative nodal positions) over absolute metric accuracy, ensuring that the generated finite element mesh remains mathematically valid for normalized analysis.

Hierarchical Validation Protocol. To ensure that the generated IR is not just syntactically valid but also physically solvable, the Validation Agent enforces a four-level audit protocol derived from expert heuristics:

- **L1 Schema Integrity:** Checks strictly against the JSON schema definitions (e.g., valid float types, non-empty arrays).
- **L2 Physical Stability:** Verifies static determinacy. For example, it detects “rigid body motion” risks by ensuring that constrained Degrees of Freedom (DoFs) cover both translation and rotation for frame structures.

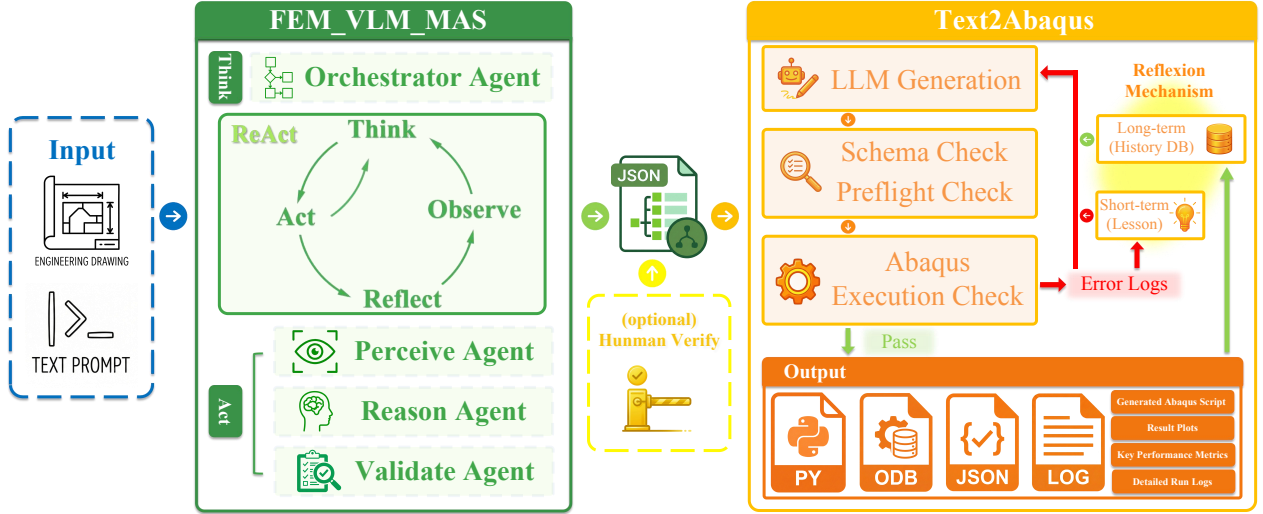


Figure 2: **The Neuro-Symbolic Architecture of VFEAgent.** The framework bridges the semantic gap between visual diagrams and physical simulation via two coupled stages: **(A) Perception**, employing a multi-agent ReAct system to extract a solver-agnostic Intermediate Representation (IR); and **(B) Synthesis**, featuring a verification-driven loop that integrates AST-based preflight checks, reflexive debugging, and a deterministic handover protocol to guarantee executability.

- **L3 Engineering Plausibility:** Audits parameter ranges (e.g., rejecting negative Poisson’s ratios or unrealistically high Young’s modulus for standard steels).
- **L4 Visual Consistency:** Performs a reverse-check to ensure that all annotated dimensions in the input image are represented in the extracted topology.

Any violation at these levels triggers an immediate feedback loop to the Reasoning Agent before the IR is finalized.

Monotonic Completeness & Context Pinning. To circumvent context drift in multi-turn dialogues, we implement an *Explicit Context Manager*. Critical engineering constraints (e.g., “Topology Optimization Mode”) are pinned to the system prompt, preventing eviction during token window sliding. Furthermore, we enforce a *Non-degenerative Acceptance Criterion*: updates to Y are rejected if they result in information entropy loss (e.g., overwriting valid nodes with empty sets) unless explicitly justified by the Validation Agent.

3.4 Stage-B: Verification-Driven Synthesis

Stage-B addresses the “Execution Gap” inherent in LLM-generated code through a verification-driven synthesis protocol, as outlined in Algorithm 1.

Preflight Static Verification. Prior to execution, the generated script S undergoes rigorous static analysis based on the Python Abstract Syntax Tree (AST). We enforce domain-specific constraints to intercept fatal errors, specifically: (1) *Lifecycle Integrity*, which verifies the presence of essential **execution triggers** and **process termination signals** to prevent idle runs; and (2) *API Safety*, which prohibits **unsafe kernel state manipulations** (e.g., deletion of protected root containers) that could compromise the solver environment.

Sandboxed Execution and Artifact Management. Executing arbitrary LLM-generated code entails significant security and state-management risks. VFEAgent implements a strict *Run Isolation* protocol, where each task is encapsulated within a dynamically allocated workspace. The generator is explicitly prompted to enforce **directory isolation** and redirect all **simulation artifacts** (including binary databases and status logs) to this secluded path. This mechanism prevents file collisions during parallel batch processing and ensures that output data is strictly associated with its parent task.

Reflexive Debugging with Experience Replay. Upon runtime failure, the Debugger Agent synthesizes structured corrective feedback. We maintain a persistent *Experience Replay Buffer* $\mathcal{H}$. For each instance, the system retrieves the top- k isomorphic failure/success patterns from $\mathcal{H}$ based on error signature similarity. This retrieval mechanism enables the agent to preemptively resolve recurrent issues (e.g., version-specific syntax incompatibilities) by leveraging historical insights.

Schema-Gated Topology Optimization. To accommodate advanced design synthesis beyond standard forward analysis, we incorporate a **Schema-Gated Branching** mechanism. Triggered by an explicit optimization intent (encoded via a dedicated semantic flag within the IR), the synthesis engine redirects the generation flow to a specialized protocol that instantiates the requisite **optimization task structures** and **solver process definitions**. To counteract the convergence instability typical of zero-shot generation, the system injects **Robust Initialization Priors** (e.g., imposing conservative volume constraints). These priors serve as operational baselines, guaranteeing solver stability while remaining adaptable to explicit user constraints extracted from the prompt.

Algorithm 1 Reflexive Synthesis with Neural-Symbolic Handover

Input: IR Y , Max Retries K , Memory $\mathcal{H}$ **Output:** Valid Script S^* , ODB Result R

```
1:  $Lessons \leftarrow \emptyset$ 
2:  $S \leftarrow \text{LLM}_{\text{Gen}}(Y, \mathcal{H})$  {Conditioned on memory}
3: for  $k = 0$  to  $K$  do
4:    $E_{\text{static}} \leftarrow \text{PreflightCheck}(S)$  {AST Analysis}
5:   if  $E_{\text{static}} \neq \emptyset$  then
6:      $Lessons \leftarrow \text{Reflect}(E_{\text{static}})$ 
7:      $S \leftarrow \text{LLM}_{\text{Repair}}(S, Lessons)$ 
8:     continue
9:   end if
10:  {Execute in isolated sandbox}
11:   $R, E_{\text{runtime}} \leftarrow \text{ExecuteSubprocess}(S, \text{env})$ 
12:  if  $E_{\text{runtime}} = \emptyset$  and  $\text{Verify}(R)$  then
13:     $\text{UpdateMemory}(\mathcal{H}, S)$  {Index success skill}
14:    return  $S, R$ 
15:  else
16:     $Lessons \leftarrow Lessons \cup \text{Debugger}(E_{\text{runtime}})$ 
17:     $S \leftarrow \text{LLM}_{\text{Repair}}(S, Lessons)$ 
18:  end if
19: end for
20:  $S_{\text{det}} \leftarrow \text{DeterministicFallback}(Y)$  {Symbolic Handover}
21:  $R \leftarrow \text{ExecuteSubprocess}(S_{\text{det}}, \text{env})$ 
22: return  $S_{\text{det}}, R$ 
```

Neural-Symbolic Handover (Deterministic Fallback).

Recognizing the stochastic nature of LLMs, we introduce a deterministic fallback mechanism. If the reflexive loop exhausts the retry budget K , the system triggers a *Symbolic Handover*, reverting to a rule-based template engine that strictly maps the IR Y to a guaranteed-executable script. This hybrid architecture ensures a baseline executability for canonical structural problems, effectively acting as a fail-safe.

4 Experiments

4.1 Experimental Setup

Implementation Details. We implement the multi-agent system using a hybrid Python runtime (v3.9 for agents, v2.7 for Abaqus kernel). For comprehensive specifications regarding the **hardware environment**, **software dependencies**, and the **structural signature retrieval algorithm**, please refer to **Appendix A**.

Benchmarks and Baselines. We evaluate VFEAgent on a comprehensive suite of engineering problems, categorized into two modes: (1) *Standard Forward Analysis*, encompassing trusses, frames, and plates under diverse boundary conditions; and (2) *Topology Optimization*, focusing on compliance minimization subject to volume constraints (e.g., $\text{volfrac} \leq 0.5$). We compare our approach against state-of-the-art multimodal LLMs, including GPT-4o, GPT-5 (Preview), Gemini-3-Pro, and Qwen-3-Max, each prompted with expert-crafted Chain-of-Thought (CoT) instructions.

Metrics. Performance is quantified using the following metrics: (1) *Schema Validity*, measuring the adherence of the intermediate output to the strict FEM JSON schema; (2) *Overall Perception Score*, a composite metric that averages

Model	Schema Valid.	Node Acc.	Conn. F1	BC Det.	Overall
VFEAgent	0.900	0.815	0.648	0.600	0.704
Gemini-3-Pro	0.417	0.775	0.577	0.458	0.639
GPT-5	0.333	0.756	0.609	0.542	0.610
GPT-4o	0.583	0.750	0.488	0.500	0.596
Gemini-3-Flash	0.333	0.752	0.576	0.458	0.583
Grok-4	0.545	0.582	0.500	0.500	0.503
Qwen-3-Max	0.091	0.127	0.024	0.136	0.128

Table 1: Interpretation performance (Stage-A). VFEAgent achieves the highest Schema Validity (90.0%), significantly outperforming baselines.

node detection accuracy and connectivity F1-score; and (3) *Execution Success Rate*, defined as the percentage of generated scripts that successfully complete the full Abaqus job lifecycle and yield a readable ODB file.

4.2 Comparative Analysis

Structural Interpretation (Stage-A). Table 1 underscores the “semantic gap” inherent in foundation models. While general-purpose VLMs such as *Gemini-3-Pro* demonstrate superior raw pixel-level perception (Node Acc. 0.815), they falter in mapping visual features to rigorous engineering schemas (Schema Validity 41.7%). VFEAgent addresses this limitation through its multi-agent ReAct loop, which enforces logical consistency constraints (e.g., ensuring that every element connects exactly two existing nodes), thereby attaining a superior validity score of 90.0%.

Execution Robustness (Stage-B). The “execution gap” is further examined in Table 2. A key insight is that superior linguistic proficiency does not necessarily equate to engineering accuracy. For example, *GPT-5* achieves a score of 1.0 on Lifecycle (accurately handling job submission) but 0.0 on Preflight Safety, largely attributable to the generation of unsafe API calls. VFEAgent’s neuro-symbolic handover protocol guarantees 100% success by reverting to deterministic templates when the probabilistic generator fails to converge. It is essential to elucidate the role of the deterministic fallback mechanism in our evaluation. Although conceived as a vital safeguard for real-world industrial applications, **this mechanism was inactive (0% activation rate) across all experimental benchmarks presented in Table 2**. The 100% execution success rate was attained solely through the neural agents employing the reflexive debugging loop ($K \leq 3$), thereby affirming the inherent robustness of our neuro-symbolic architecture without dependence on rule-based templates in these standard scenarios.

4.3 Taxonomy of LLM Engineering Failures

Beyond success rates, we categorize the dominant failure modes of foundation models in CAE tasks, derived from the error logs of Experiment 2.

Type I: Lifecycle Blindness (45% of failures). Foundation models often treat the script as a static description rather than a dynamic process. A frequent error involves defining the entire model correctly but omitting the `job.submit()`

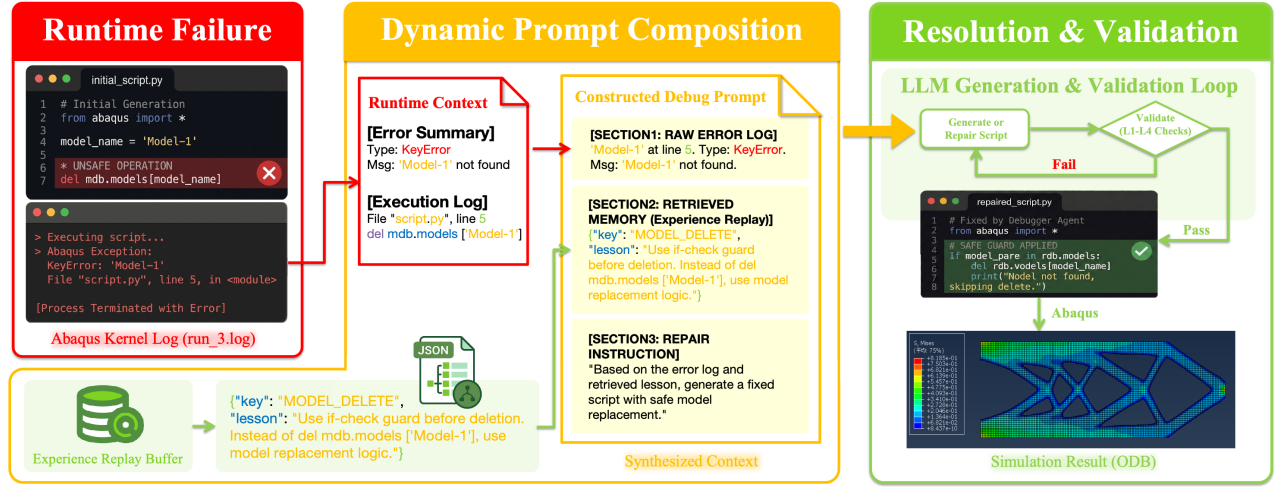


Figure 3: **Case Study: Autonomous Recovery via Composite Context Debugging.** (A) **The Crash:** The script attempted an unsafe root model deletion, triggering a `KeyError`. (B) **Context Assembly (The "Hard" Part):** Instead of simple error matching, the Debugger Agent constructs a composite prompt. It aggregates the **Structured Error Summary** (for high-level intent) and the **Raw Execution Log** (for line-level localization). Simultaneously, it retrieves a specific *Experience Lesson* (JSON) to ground the repair in domain rules. (C) **Verified Fix:** Armed with both the specific traceback and the domain constraint, the Repair Agent synthesizes a precise `try-except` guard, successfully resolving the crash.

Model	Exec. Preflight	Safety Life.	Help.	Physical Comp. BC Load Avg.		
Qwen-3-Max	0.50	1.0	1.0	0.69	0.37	0.53
Grok-4	0.50	1.0	1.0	0.69	0.37	0.53
Gemini-3-Pro	0.25	0.5	1.0	0.38	0.30	0.34
GPT-5	0.00	1.0	0.75	0.81	0.39	0.60
Ours	1.00	1.0	1.0	1.00	1.00	1.00

Table 2: **Granular Analysis (Stage-B).** Breakdown of generation quality. **Ours** guarantees 100% compliance across all safety and physical metrics.

or `waitForCompletion()` calls. This results in an “idle run” that terminates without triggering the solver kernel, thereby producing no ODB data.

Type II: Contextual API Hallucination (30% of failures). Models often invent non-existent methods by conflating different object levels. For example, we observed models calling `mesh.EdgeArray(...)`—a method that does not exist in the Abaqus API—instead of accessing edges via the Part object. This reflects a lack of fine-grained knowledge about the hierarchical Abaqus Object Model (AOM).

Type III: Unsafe State Manipulation (25% of failures). To ensure a “clean slate,” models frequently attempt to execute `del mdb.models['Model-1']`. In the Abaqus CAE environment, ‘Model-1’ is often the protected root container, and deleting it causes an immediate kernel crash. This nuance is rarely documented in public training corpora, rendering it a common zero-shot failure mode.

4.4 Qualitative Analysis: Self-Correction in Action

Figure 3 presents a detailed execution log, demonstrating the system’s robust debugging capability.

The Runtime Failure (Panel A). The Executor Agent attempted to sanitize the environment using `del mdb.models['Model-1']`. While valid in standard Python, this operation violates the Abaqus kernel’s protection of the root node, resulting in a fatal `KeyError`.

Dynamic Context Composition (Panel B). The core of our debugging engine lies not merely in retrieving a fix, but in constructing an **information-complete context** for the LLM. As detailed in the prompt protocols (Appendix B), the Debugger Agent performs a multi-source aggregation:

- Log Parsing:** It captures the full *execution traceback* to localize the fault line (Line 45) and generates a structured *error summary* (e.g., Invalid Dictionary Key Access”).
- Knowledge Retrieval:** It uses the error signature to query the Experience Replay Buffer, retrieving the crucial domain constraint: *Root model deletion is forbidden.*”

This composite input ensures that the model understands not just *what* failed (the error), but also *where* (the log) and *how* to fix it (the lesson).

Successful Resolution (Panel C). Guided by this rich context, the Repair Agent implemented a defensive `try-except` block. The script successfully converged (15 iterations), confirming that feeding raw runtime logs alongside retrieved knowledge is essential for resolving “Type III” engineering failures.

4.5 Discussion and Limitations

While VFEAgent demonstrates strong capabilities in automating standard structural analysis, we identify two primary limitations:

Dependence on Visual Fidelity. The system’s performance is currently bounded by the resolution of the upstream VLM. In cases of low-resolution blueprints where dimension lines overlap with structural members, the Perception Agent may hallucinate connectivity. Future work will integrate vector-graphics parsing (SVG) to bypass raster-level ambiguity.

3D Reconstruction Ambiguity. Extracting full 3D depth from a single 2D orthographic view remains an ill-posed problem. Our current “Constraint-Aware” estimation relies on standard engineering conventions (e.g., extruding 2D profiles). Handling complex non-manifold 3D geometries would require multi-view reasoning capabilities, which represents a direction for our next-generation framework.

5 Conclusion

In this paper, we introduce VFEAgent, a neuro-symbolic multi-agent framework that automates the end-to-end workflow of finite element analysis. By decomposing the complex modeling task into a ReAct-driven visual perception stage and a verification-first synthesis stage, VFEAgent effectively bridges the “semantic gap” in interpreting engineering diagrams and the “execution gap” in generating rigorous simulation scripts.

Our empirical evaluations indicate that large language models, when augmented with hierarchical validation and reflexive debugging, can achieve near-industrial-grade reliability. VFEAgent demonstrated robust performance across the evaluated structural benchmarks, yielding valid simulation outcomes solely through neural self-correction ($K \leq 3$) in the tested scenarios. Moreover, the automated workflow exhibited a substantial efficiency gain, reducing modeling latency from typical manual durations (minutes) to near-real-time execution (seconds), thereby underscoring its potential to streamline iterative design processes in computational engineering.

Future work will address the inherent limitations of 2D raster inputs by incorporating 3D-native perception modalities (e.g., point clouds or STEP files) to manage complex non-manifold geometries. Additionally, we plan to extend the framework’s solver-agnostic intermediate representation to support open-source engines such as OpenSees and CalculiX, thereby further democratizing access to autonomous simulation technologies.

References

- [Azanaw, 2025] Girmay Mengesha Azanaw. Beyond the classical grid: A critical review of emerging finite element methodologies in structural engineering—from AI-enhanced solvers to isogeometric, discontinuous, and phase-field paradigms. *ResearchGate Preprint*, July 2025. Preprint, not peer-reviewed.
- [Baker et al., 2025] Christopher Baker, Karen Rafferty, and Mark Price. Large language models in mechanical engineering: A scoping review of applications, challenges, and future directions. *Big Data and Cognitive Computing*, 9(12):305, 2025.
- [Fan et al., 2025] E Fan, Kang Hu, Zhuowen Wu, Jiangyang Ge, Jiawei Miao, Yuzhi Zhang, He Sun, Weizong Wang, and Tianhan Zhang. ChatCFD: An LLM-driven agent for end-to-end CFD automation with domain-specific structured reasoning. *arXiv preprint arXiv:2506.02019*, 2025.
- [Gembariski, 2020] Paul Christoph Gembariski. Agent collaboration in a multi-agent-system for analysis and optimization of mechanical engineering parts. In *Knowledge-Based and Intelligent Information & Engineering Systems*, volume 176, pages 592–601. Elsevier, 2020.
- [Geng et al., 2025] Ziheng Geng, Jiachen Liu, Ran Cao, Lu Cheng, Haifeng Wang, and Minghui Cheng. A lightweight large language model-based multi-agent system for 2D frame structural analysis. *arXiv preprint arXiv:2510.05414*, 2025.
- [Ghorbani et al., 2024] Javad Ghorbani, Ramin Aghdasi, Mingxiang Li, Chenguang Zhou, Yongming Li, Wei Zhang, and Liang Chen. Parameters in play: Alphazero-inspired AI for autonomous parameter identification in soil constitutive and finite element models. *Computers and Geotechnics*, 166:106065, 2024.
- [G”opfert et al., 2023] Jan G”opfert, Jann M. Weinand, Patrick Kuckertz, and Detlef Stolten. Opportunities for large language models and discourse in engineering design. *arXiv preprint arXiv:2306.09169*, 2023.
- [Green et al., 2024] M. D. Green, K. S. Kirilov, M. Turner, J. Marcon, J. Eichstädt, E. Laughton, C. D. Cantwell, S. J. Sherwin, J. Peiró, and D. Moxey. NekMesh: An open-source high-order mesh generation framework. *Computer Physics Communications*, 298:109089, 2024.
- [Guo et al., 2025] Xingang Guo, Yaxin Li, Xiangyi Kong, Yilan Jiang, Xiayu Zhao, Zhichao Zhang, Zhichao Sun, Xinyu Wang, Yuhang Li, Yifan Li, et al. Toward engineering AGI: Benchmarking the engineering design capabilities of LLMs. *arXiv preprint arXiv:2509.16204*, 2025.
- [Guru et al., 2025] Mahish K. Guru, Vipul Gupta, Kartik Bali, and Roland Aydin. From finite element analysis to self-optimization: FRAME (Finite element Reasoning and AI-Agent Model Engine) and the era of agentic AI in engineering. Under review as a conference paper at ICLR 2025, 2025. Available at OpenReview: <https://openreview.net/forum?id=2TdYc5V4jU>.
- [Hong et al., 2024] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Wang, Jialin Cheng, Jinlin Zhang, Ceyao Wang, Ziyang Yau, Steven Ka Shing Lin, Liyang Zhou, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- [Hou et al., 2025] Shifu Hou, Rick Johnson, Ramandeep Makhija, Lingwei Chen, and Yanfang Ye. AutoFEA: Enhancing AI copilot by integrating finite element analysis

- using large language models with graph neural networks. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence*, volume 39, pages 24078–24085, 2025.
- [Khan *et al.*, 2024] Muhammad Tayyab Khan, Lequn Chen, Ye Han Ng, Wenhe Feng, Nicholas Yew Jin Tan, and Seung Ki Moon. Fine-tuning vision-language model for automated engineering drawing information extraction. *arXiv preprint arXiv:2411.03707*, 2024.
- [Khan *et al.*, 2025] Muhammad Tayyab Khan, Lequn Chen, Zane Yong, Jun Ming Tan, Wenhe Feng, and Seung Ki Moon. From drawings to decisions: A hybrid vision-language framework for parsing 2D engineering drawings into structured manufacturing knowledge. *Robotics and Computer-Integrated Manufacturing*, 91:102862, 2025. *arXiv preprint arXiv:2506.17374*.
- [Liu *et al.*, 2022] Wing Kam Liu, Shaofan Li, and Harold S. Park. Eighty years of the finite element method: Birth, evolution, and future. *Archives of Computational Methods in Engineering*, 29(6):4431–4497, 2022.
- [Mohammadzadeh *et al.*, 2025] Saeed Mohammadzadeh, Erfan Hamdi, Joel Shor, and Emma Lejeune. FEM-Bench: A structured scientific reasoning benchmark for evaluating code-generating LLMs. *arXiv preprint arXiv:2512.20732*, 2025.
- [Mudur *et al.*, 2025] Nayantara Mudur, Hao Cui, Subhashini Venugopalan, Paul Raccuglia, Michael P. Brenner, and Peter Norgaard. FEABench: Evaluating language models on multiphysics reasoning ability. *arXiv preprint arXiv:2504.06260*, 2025. Under review as a conference paper at ICLR 2025.
- [Ni and Buehler, 2024] Bo Ni and Markus J. Buehler. MechAgents: Large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge. *Extreme Mechanics Letters*, 67:102131, 2024.
- [Qi *et al.*, 2025] Yupeng Qi, Ran Xu, and Xu Chu. FeaGPT: an end-to-end agentic-AI for finite element analysis. *arXiv preprint arXiv:2510.21993*, 2025.
- [Sahani, 2025] Suresh Kumar Sahani. Ai-enhanced finite element method (FEM) for structural analysis. *Journal of Electrical Systems*, 20(1):661–676, 2025.
- [Shah *et al.*, 2024] Vatsal Shah, Rajesh Singh Thakur, Rohit Mishra, Anurag Gupta, Rajendra Kumar Singh, and Sandeep Kumar Mishra. Application of machine learning and deep learning in finite element analysis: A comprehensive review. *Archives of Computational Methods in Engineering*, 31(3):1413–1451, 2024.
- [Shinn *et al.*, 2024] Noah Shinn, Federico Cassano, Ashwin Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2024.
- [Tian and Zhang, 2024] Chuan Tian and Yilei Zhang. Collaboration dynamics and reliability challenges of multi-agent LLM systems in finite element analysis. *arXiv preprint arXiv:2408.13406*, 2024.
- [Wei *et al.*, 2022] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, fei bfe, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.
- [Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [Zhang *et al.*, 2025] Tao Zhang, Zhenhai Liu, Yong Xin, and Yongjun Jiao. MooseAgent: A LLM based multi-agent framework for automating MOOSE simulation. *arXiv preprint arXiv:2504.08621*, 2025.

A Implementation Details

Hardware & Computing Environment. To ensure a rigorous evaluation of inference latency, all baselines and our VFEAgent system were evaluated on the same high-performance workstation. The testbed is equipped with dual **AMD EPYC 9654** processors (384 threads) and 512 GB of RAM. The simulation engine is powered by **Simulia Abaqus 2022** running on a Linux environment. This standardized hardware setup eliminates performance variances caused by differing compute resources.

Software Architecture. The multi-agent system is orchestrated via the LangChain framework. To bridge the compatibility gap between modern AI agents and legacy CAE software, we employ a dual-environment architecture: the agents operate in a Python 3.9 runtime, while the simulation execution and ODB extraction are handled by an isolated Python 2.7 sandbox strictly compatible with the Abaqus Scripting Interface.

Structural Signature Retrieval. Instead of relying on opaque vector embeddings, we implement a transparent **Heuristic Structural Signature** matching algorithm for experience replay. We define a model’s signature as a tuple $S = (N_{node}, N_{elem}, \{T_{sec}\}, N_{load}, N_{bc})$. During runtime debugging, the system retrieves historical success cases where the node-to-element density ratio falls within a calibrated tolerance interval $[0.5, 2.0]$. This engineering-centric metric ensures that retrieved lessons are topologically relevant to the current failure mode.